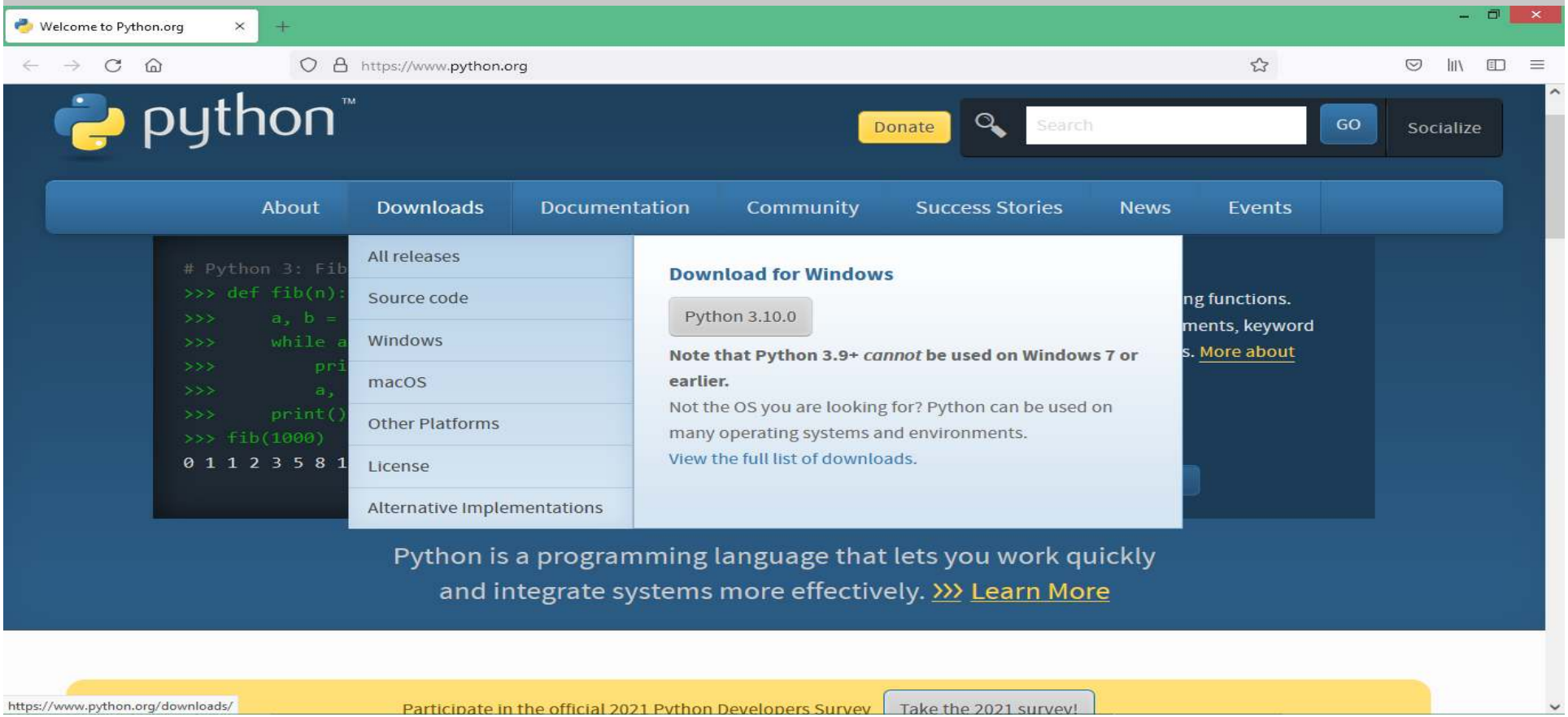
Programowanie obliczeń komputerowych

Wprowadzenie do środowiska interpretacji skryptów języka Python

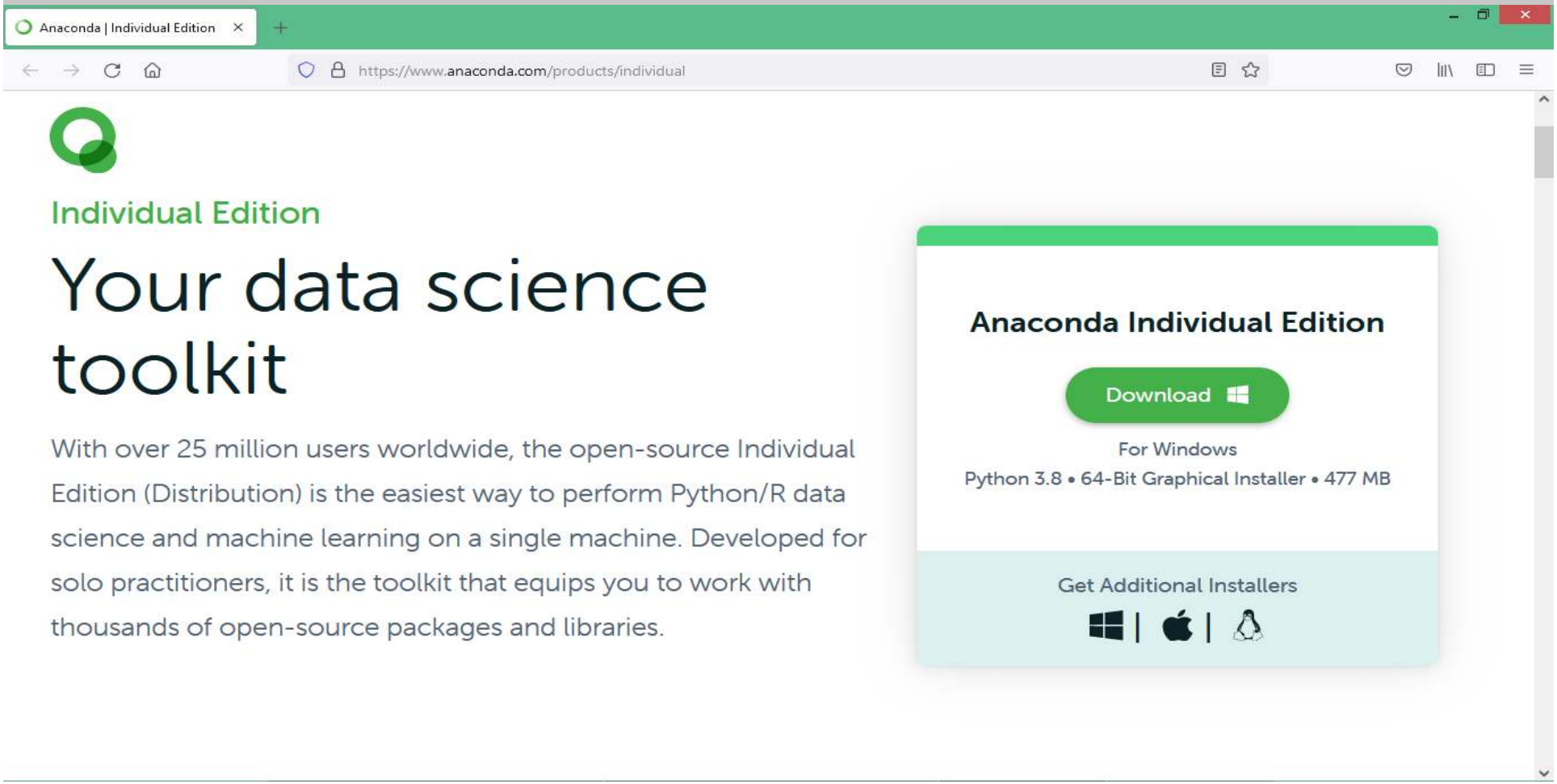
Dystrybucje środowiska Python

- www.python.org: oficjalny serwis głównej dystrybucji środowiska skryptowego języka Python z darmowym dostępem do otwartego oprogramowania źródłowego w języku C/C++ oraz binarnego dla systemu Windows i MacOS oraz innych platform, np..: AIX, iOS, Solaris, VMS oraz dla różnych dystrybucji systemu Linux



Dystrybucje środowiska Python

- www.anaconda.com: serwis komercyjnej dystrybucji środowiska skryptowego języka Python zorientowanej na badania naukowe z darmowym dostępem do edycji indywidualnej w ramach otwartego oprogramowania przeznaczonego dla wielu platform systemowych



Inne dystrybucje darmowe

- winpython.github.io: dystrybucja środowiska Python dla systemu Windows zorientowana na tworzenie aplikacji skryptowych z graficznym interfejsem użytkownika
- ironpython.net: dystrybucja środowiska Python dla platformy .NET zorientowana na integrację ze oprogramowaniem języka C# oraz na programowanie dynamiczne w ramach podsystemu DLR
- www.sagemath.org: skryptowe środowisko przeznaczone do obliczeń matematycznych w ramach algebry komputerowej z graficznym interfejsem notatnikowym opartym na języku Python
- pytorch.org: skryptowe środowisko dot. uczenia maszynowego
- www.spyder-ide.org: zintegrowane środowisko programowania naukowego w języku Python oferujące narzędzia oraz edytor z graficznym interfejsem użytkownika
- pandas.pydata.org: zestaw narzędzi do analizy i przetwarzania danych oparty na języku Python **(…)**

Podstawowa instalacja środowiska Python

- **python-3.10.4.exe**: instalator najnowszej wersji dla języka Python3 oraz dla 32-bitowej platformy x86 systemu Windows od wersji 8
- **python-3.10.4-amd64.exe**: rekomendowana instalacja dla trybu 64-bitowego systemu Windows

Uwaga: Python 3.6 nie może być instalowany w systemach Windows XP i mniej

Uwaga: Python 3.9 nie może być instalowany w systemach Windows 7 i mniej

- **python-2.7.18.msi**: instalator ostatniej wersja dla języka Python2 nadający się dla starszych 32-bitowych systemów Windows od NT4
- W trakcie biegu instalatora należy skonfigurować instalację, gdzie wybieramy opcjonalne pakiety i własności oraz podajemy ścieżkę do głównego katalogu <**pyroot**>, np.: **C:\Python27**, **C:\Python310**
- Można opcjonalnie dodać tą ścieżkę do zmiennej PATH w systemie Windows oraz ścieżkę do podkatalogu: **<pyroot>\Scripts**
- W katalogach tych znajdują się główne narzędzia środowiska
- Inne moduły i pakiety znajdują się w podkatalogu: **<pyroot>\Lib**

Instalacja dodatkowych pakietów

- **<pyroot>\Scripts\pip.exe**: wygodne narzędzie do instalacji extra pakietów z oficjalnych repozytoriów środowiska Python
- pypi.org: spis wielu opublikowanych pakietów środowiska Python
- www.lfd.uci.edu/~gohlke/pythonlibs: nieoficjalne repozytorium pakietów przeznaczonych jedynie dla systemu Windows
- Przykład instalacji pakietów IPython jako rozszerzenie środowiska o interaktywny interfejs użytkownika z wizualizacją graficzną w Qt:

```
pip install pyqt5
pip install "ipython[all]"
```

- Przykład instalacji notatnikowego interfejsu użytkownika w IE:

```
pip install jupyterlab
pip install "nbconvert[all]"
```

- Przykład instalacji pakietów naukowych:

```
pip install scipy-stack
```

Interpreter Python i powłoki użytkownika

- **<pyroot>\python.exe**: standardowy interpreter języka i zarządca środowiska skryptowego; użycie w trybie wsadowym poprzez linię poleceń - uruchomienie bez argumentów daje tryb konwersacyjny w oknie konsoli (**conhost.exe**) jako powłokę z linią interpretacji „>>>”

Uwaga: W nazwie tego programu też można wstawić wersję instalacji, np.: python310.exe

- **<pyroot>\pythonw.exe**: wersja dla systemu Windows umożliwia wyświetlanie okien graficznych GUI z skryptów *.pyw w powłoce
- **<pyroot>\Lib\...\idle***: extra powłoka z linią interpretacji skryptu zrealizowana w oknie GUI o menu poleceń jako proste Python IDE
- **<pyroot>\Scripts\ipython***: osobna powłoka ze pełną interakcją i graficzną wizualizacją w osobnym panelu Qt lub liniowo w formie notatnika - skrypt+grafika; wyświetla linie interpretacji skryptu jako wejście interpretera „In [1]” oraz wyjście interpretera „Out [1]”
- **<pyroot>\Scripts\jupyter-notebook***: najnowsza złożona powłoka notatnikowa w przeglądarce internetowej z interakcją i wizualizacją
- **<pyroot>\Scripts\spyder***: podobna powłoka w oknie IDE **(…)**

Najprostsze operacje w środowisku

- Interpretacja skryptu języka Python w podanym wejściowym pliku wsadu *.py z opcjonalnie podanymi argumentami skryptu, np.:

```
python [-i] skrypt.py abc 123
```

- Interpretacja skryptu wsadu w linii poleceń, np.:

```
python310 [-i] -c "print(123)"
```

opcja [-i] daje przejście do trybu powłoki

- Wykonanie zainstalowanego w środowisku narzędziowego lub podanego modułu skryptowego z pliku *.py w linii poleceń, np.:

```
python -m pip install scipy-stack
python [-i] [-O] -m skrypt
```

opcja [-O] daje optymalizację

- W linii interpretacji standardowej powłoki uzyskanie pomocy:

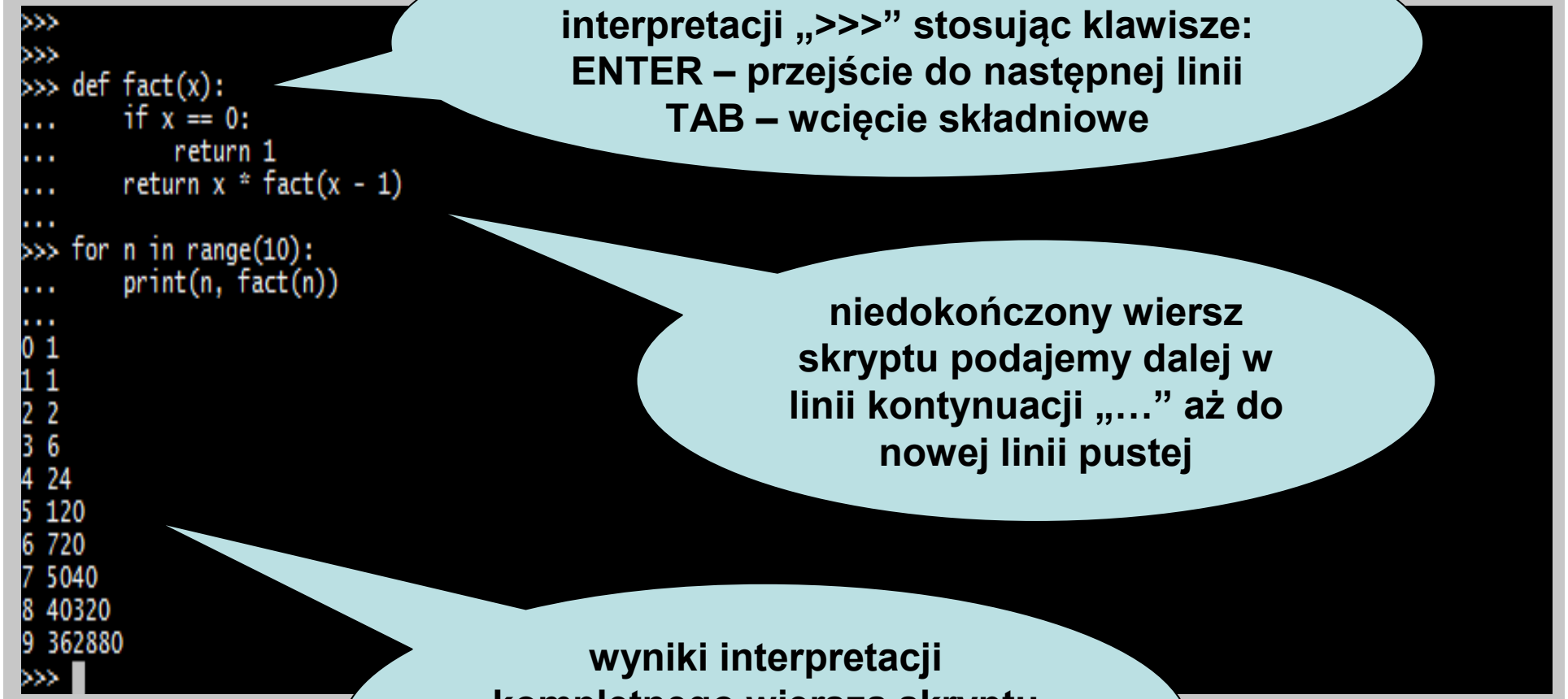
```
>>> help()
help> modules
```

wyświetlenie listy modułów zainstalowanych w środowisku

- Wyjście z powłoki standardowej i powrót do systemu:

```
>>> quit()
```

Przykład skryptu w powłoce standardowej



Przykład skryptu w powłoce notatnikowej

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\adam

IPython console

Console 1/A

```
In [1]: import matplotlib.pyplot as plt

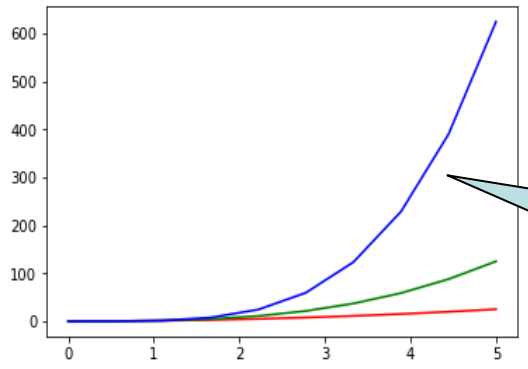
In [2]: import numpy as np

In [3]: x = np.linspace(0, 5, 10)

In [4]: y = x ** 2

In [5]: 123*456+789
Out[5]: 56877

In [6]: plt.plot(x, y, 'r', x, x ** 3, 'g', x, x ** 4, 'b')
Out[6]:
[<matplotlib.lines.Line2D at 0xc0e66d8>,
 <matplotlib.lines.Line2D at 0xbdf9080>,
 <matplotlib.lines.Line2D at 0xc0e6e10>]
```




```
In [7]:
```

skrypt wejściowy w kolejnych liniach interpretacji

tekstowe wyniki na wyjściu z interpretera

graficzne obiekty na wyjściu z interpretera

wyniki graficzne wyświetlane są liniowo obok skryptu i innych tekstów co daje interakcję w formie notatnika

Python console | History log | IPython console

Permissions: RW | End-of-lines: CRLF | Encoding: UTF-8 | Line: 1 | Column: 1 | Memory: 47 %

Przykład zwykłej powłoki interaktywnej

File Edit View Kernel Magic Window Help

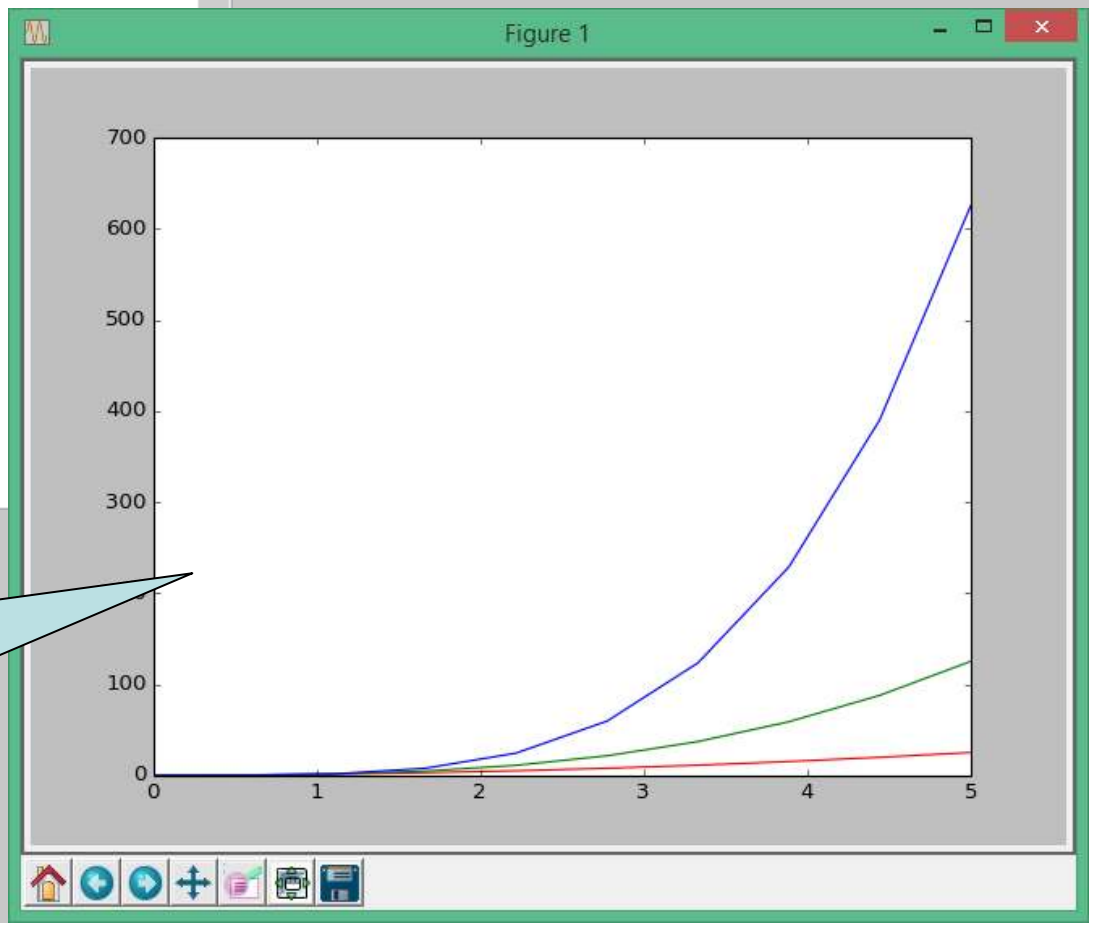
```
In [1]: import matplotlib.pyplot as plt
   ...: import numpy as np
   ...:
   ...: x = np.linspace(0, 5, 10)
   ...: y = x ** 2
   ...: plt.plot(x, y, 'r', x, x ** 3, 'g', x, x ** 4, 'b')
   ...:
Out[1]:
[<matplotlib.lines.Line2D at 0x4ecdb50>,
 <matplotlib.lines.Line2D at 0x4ecdd50>,
 <matplotlib.lines.Line2D at 0x4ed9270>]

In [2]: plt.show()
```

skrypt wejściowy w liniach interpretacji i kontynuacji

graficzne obiekty na wyjściu wymagają podania jawnej operacji do ich wizualizacji

wyniki graficzne wyświetlane są w osobnym panelu Qt z paskiem poleceń u dołu okna



Charakterystyka języka Python

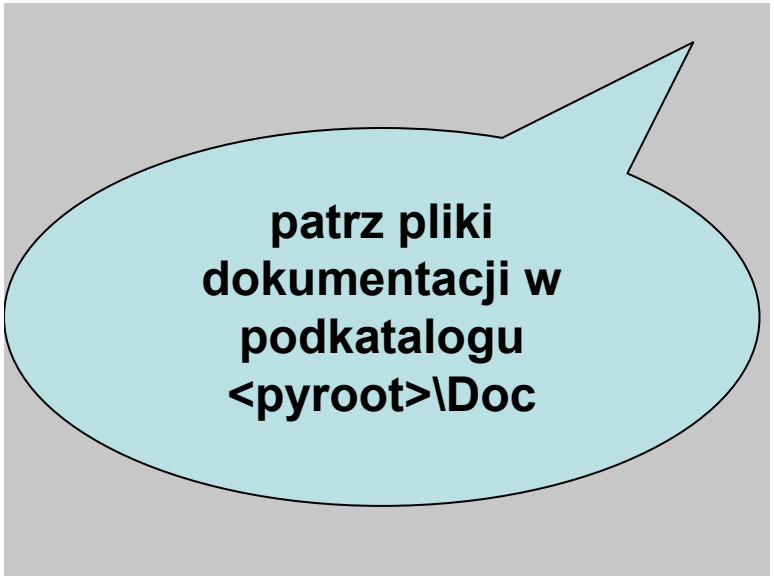
- Stworzony przez Guido van Rossum w 1990 r.
- Nazwa pochodzi od serialu telewizyjnego *Monty Python*
- Główne wersje, 1.0: 1990r, 2.0: 2000r, 3.0: 2008r; obecnie: 3.10
- Język interpretowany – brak maszynowej kompilacji i konsolidacji
- W pełni dynamiczny - brak typów zmiennych i jawnej deklaracji typu; typy posiadają wewnętrznie jedynie dane w skrypcie
- Opiera się na zarządzanej stercie z automatyczną alokacją pamięci i kolektorem nieużytków GC (ang. *garbage collector*)
- Wspiera programowanie proceduralne, obiektowe i funkcyjne
- Umożliwia przetwarzanie złożonych struktur danych przy użyciu operacji wysokiego poziomu
- Umożliwia organizację kodu w modułach, klasach i procedurach
- Umożliwia wysyłanie i obsługę wyjątków
- Posiada przejrzystą składnie opartą o formatowanie tekstu

Charakterystyka środowiska operacyjnego

- Zawiera standardowe narzędzia i bibliotekę (pakiety z modułami)
- Oprogramowanie systemowe napisane w języku C z użyciem systemowego interfejsu API początkowo jedynie dla Linux
- Możliwość nieograniczonego rozszerzania poprzez tworzenie w C własnych modułów binarnych *.pyd ~ .dll|so opartych o interfejs CPython API oraz interfejsy innych języków (C#, Java)
- Umożliwia swobodną integrację oprogramowania różnych języków
- Oferuje wspólny interfejs skryptowy użytkownika środowiska oparty na własnym języku uniwersalnym Python
- Posiada wbudowany interpreter REPL (ang. *read-eval-print-loop*)
- Kompiluje moduł *.py do kodu beta *.pyc z optymalizacją w *.pyo
- Zarządza pamięcią dynamiczną oraz innymi zasobami systemu niezbędnymi do wykonania skryptu - maszyna wirtualna Python
- Oferuje dostęp w skryptach do różnych aspektów funkcjonalności systemu i aplikacji (rejestry, pliki, sieć, grafika, bazy danych, …)

Biblioteka standardowa Python

- Text Processing Services
  - `string` — Common string operations
  - `re` — Regular expression operations
  - `difflib` — Helpers for computing deltas
  - `textwrap` — Text wrapping and filling
  - `unicodedata` — Unicode Database
  - `stringprep` — Internet String Preparation
  - `readline` — GNU readline interface
  - `rlcompleter` — Completion function fo
- Binary Data Services
  - `struct` — Interpret bytes as packed bin
  - `codecs` — Codec registry and base clas



- Data Types
  - `datetime` — Basic date and time types
  - `zoneinfo` — IANA time zone support
  - `calendar` — General calendar-related functions
  - `collections` — Container datatypes
  - `collections.abc` — Abstract Base Classes for Containers
  - `heapq` — Heap queue algorithm
  - `bisect` — Array bisection algorithm
  - `array` — Efficient arrays of numeric values
  - `weakref` — Weak references
  - `types` — Dynamic type creation and names for built-in types
  - `copy` — Shallow and deep copy operations
  - `pprint` — Data pretty printer
  - `reprlib` — Alternate `repr()` implementation
  - `enum` — Support for enumerations
  - `graphlib` — Functionality to operate with graph-like structures

Biblioteka standardowa Python

- Numeric and Mathematical Modules
  - `numbers` — Numeric abstract base classes
  - `math` — Mathematical functions
  - `cmath` — Mathematical functions for complex numbers
  - `decimal` — Decimal fixed point and floating point arithmetic
  - `fractions` — Rational numbers
  - `random` — Generate pseudo-random num
  - `statistics` — Mathematical statistics fu
- Functional Programming Modules
  - `itertools` — Functions creating iterator
  - `functools` — Higher-order functions and
  - `operator` — Standard operators as funct
- File and Directory Access
  - `pathlib` — Object-oriented filesystem paths
  - `os.path` — Common pathname manipulations
  - `fileinput` — Iterate over lines from multiple input streams
  - `stat` — Interpreting `stat()` results
  - `filecmp` — File and Directory Comparisons
  - `tempfile` — Generate temporary files and directories
  - `glob` — Unix style pathname pattern expansion
  - `fnmatch` — Unix filename pattern matching
  - `linecache` — Random access to text lines
  - `shutil` — High-level file operations
- Data Persistence
  - `pickle` — Python object serialization
  - `copyreg` — Register `pickle` support functions
  - `shelve` — Python object persistence
  - `marshal` — Internal Python object serialization
  - `dbm` — Interfaces to Unix “databases”
  - `sqlite3` — DB-API 2.0 interface for SQLite databases

Biblioteka standardowa Python

- File Formats
  - csv — CSV File Reading and Writing
  - configparser — Configuration file parser
  - netrc — netrc file processing
  - xdrlib — Encode and decode XDR data
  - plistlib — Generate and parse Apple .plist files
- Cryptographic Service
  - hashlib — Secur
  - hmac — Keyed-Ha
  - secrets — Gener
- Generic Operating System Services
  - os — Miscellaneous operating system interfaces
  - io — Core tools for working with streams
  - time — Time access and conversions
  - argparse — Parser for command-line options, arguments and sub-commands
  - getopt — C-style parser for command line options
  - logging — Logging facility for Python
  - logging.config — Logging configuration
  - logging.handlers — Logging handlers
  - getpass — Portable password input
  - curses — Terminal handling for character-cell displays
  - curses.textpad — Text input widget for curses programs
  - curses.ascii — Utilities for ASCII characters
  - curses.panel — A panel stack extension for curses
  - platform — Access to underlying platform's identifying data
  - errno — Standard errno system symbols
  - ctypes — A foreign function library for Python

Biblioteka standardowa Python

- Concurrent Execution
  - `threading` — Thread-based parallelism
  - `multiprocessing` — Process-based parallelism
  - `multiprocessing.shared_memory` — Provides shared memory for direct access across processes
  - The `concurrent` package
  - `concurrent.futures` — Launching parallel tasks
  - `subprocess` — Subprocess management
  - `sched` — Event scheduler
  - `queue` — A synchronized queue class
  - `contextvars` — Context Variables
  - `_thread` — Low-level threading API
- Networking and Interprocess Communication
  - `asyncio` — Asynchronous I/O
  - `socket` — Low-level networking interface
  - `ssl` — TLS/SSL wrapper for socket objects
  - `select` — Waiting for I/O completion
  - `selectors` — High-level I/O multiplexing
  - `asyncore` — Asynchronous socket handler
  - `asynchat` — Asynchronous socket command/response handler
  - `signal` — Set handlers for asynchronous events
  - `mmap` — Memory-mapped file support

Biblioteka standardowa Python

- Internet Data Handling
  - email — An email and MIME handling package
  - json — JSON encoder and decoder
  - mailcap — Mailcap file handling
  - mailbox — Manipulate mailboxes in various formats
  - mimetypes — Map filenames to MIME types
  - base64 — Base16, Base32, Base64, Base85 Data Encodings
  - binhex — Encode and decode binhex4 files
  - binascii — Convert between binary and ASCII
  - quopri — Encode and decode MIME quoted-printable data
  - uu — Encode and decode uuencode files
- Structured Markup Processing Tools
  - html — HyperText Markup Language support
  - html.parser — Simple HTML and XHTML parser
  - html.entities — Definitions of HTML general entities
  - XML Processing Modules
  - xml.etree.ElementTree — The ElementTree XML API
  - xml.dom — The Document Object Model API
  - xml.dom.minidom — Minimal DOM implementation
  - xml.dom.pulldom — Support for building partial DOM trees
  - xml.sax — Support for SAX2 parsers
  - xml.sax.handler — Base classes for SAX handlers
  - xml.sax.saxutils — SAX Utilities
  - xml.sax.xmlreader — Interface for XML parsers

Biblioteka standardowa Python

- Internet Protocols and Support
  - `webbrowser` — Convenient web-browser controller
  - `cgi` — Common Gateway Interface support
  - `cgitb` — Traceback manager for CGI scripts
  - `wsgiref` — WSGI Utilities and Reference Implementation
  - `urllib` — URL handling modules
  - `urllib.request` — Extensible library for opening URLs
  - `urllib.response` — Response classes used by urllib
  - `urllib.parse` — Parse URLs into components
  - `urllib.error` — Exception classes raised by urllib.request
  - `urllib.robotparser` — Parser for robots.txt
  - `http` — HTTP modules
  - `http.client` — HTTP protocol client
  - `ftplib` — FTP protocol client
  - `poplib` — POP3 protocol client
  - `imaplib` — IMAP4 protocol client
  - `nntplib` — NNTP protocol client
  - `smtplib` — SMTP protocol client
  - `smtpd` — SMTP Server
  - `telnetlib` — Telnet client
  - `uuid` — UUID objects according to **RFC 4122**
  - `socketserver` — A framework for network servers

Biblioteka standardowa Python

- Multimedia Services
  - audioop — Manipulate raw audio data
  - aifc — Read and write AIFF and AIFC files
  - sunau — Read and write Sun AU files
  - wave — Read and write WAV files
  - chunk — Read IFF chunked data
  - colorsys — Conversions between color systems
  - imghdr — Determine the type of an image
  - sndhdr — Determine type of sound file
  - ossaudiodev — Access to OSS-compatible a
- Internationalization
  - gettext — Multilingual internationalization ser
  - locale — Internationalization services

- Program Frameworks
  - turtle — Turtle graphics
  - cmd — Support for line-oriented command interpreters
  - shlex — Simple lexical analysis
- Graphical User Interfaces with Tk
  - tkinter — Python interface to Tcl/Tk
  - tkinter.colorchooser — Color choosing dialog
  - tkinter.font — Tkinter font wrapper
  - Tkinter Dialogs
  - tkinter.messagebox — Tkinter message prompts
  - tkinter.scrolledtext — Scrolled Text Widget
  - tkinter.dnd — Drag and drop support
  - tkinter.ttk — Tk themed widgets
  - tkinter.tix — Extension widgets for Tk
  - IDLE

Biblioteka standardowa Python

- Development Tools
  - `typing` — Support for type hints
  - `pydoc` — Documentation generator and online help system
  - Python Development Mode
  - Effects of the Python Development Mode
  - ResourceWarning Example
  - Bad file descriptor error example
  - `doctest` — Test interactive Python examples
  - `unittest` — Unit testing framework
  - `unittest.mock` — mock object library
  - `unittest.mock` — getting started
  - 2to3 - Automated Python 2 to 3 code translation
  - `test` — Regression tests package for Python
  - `test.support` — Utilities for the Python test suite
  - `test.support.socket_helper` — Utilities for socket tests
  - `test.support.script_helper` — Utilities for the Python execution tests
  - `test.support.bytecode_helper` — Support tools for testing correct bytecode generation
  - `test.support.threading_helper` — Utilities for threading tests
  - `test.support.os_helper` — Utilities for os tests
  - `test.support.import_helper` — Utilities for import tests
  - `test.support.warnings_helper` — Utilities for warnings tests

Biblioteka standardowa Python

- Debugging and Profiling
  - Audit events table
  - bdb — Debugger framework
  - faulthandler — Dump the Python traceback
  - pdb — The Python Debugger
  - The Python Profilers
  - timeit — Measure execution time
  - trace — Trace or track Python sta
  - tracemalloc — Trace memory all
- Software Packaging and Distribution
  - distutils — Building and installi
  - ensurepip — Bootstrapping the p
  - venv — Creation of virtual environ
  - zipapp — Manage executable Pyt
- Python Runtime Services
  - sys — System-specific parameters and functions
  - sysconfig — Provide access to Python's configuration information
  - builtins — Built-in objects
  - __main__ — Top-level code environment
  - warnings — Warning control
  - dataclasses — Data Classes
  - contextlib — Utilities for with-statement contexts
  - abc — Abstract Base Classes
  - atexit — Exit handlers
  - traceback — Print or retrieve a stack traceback
  - __future__ — Future statement definitions
  - gc — Garbage Collector interface
  - inspect — Inspect live objects
  - site — Site-specific configuration hook
- Custom Python Interpreters
  - code — Interpreter base classes
  - codeop — Compile Python code

Biblioteka standardowa Python

- Importing Modules
  - zipimport — Import modules from Zip archives
  - pkgutil — Package extension utility
  - modulefinder — Find modules used by a script
  - runpy — Locating and executing Python modules
  - importlib — The implementation of
  - Using importlib.metadata
- Python Language Services
  - ast — Abstract Syntax Trees
  - symtable — Access to the compiler's
  - token — Constants used with Python
  - keyword — Testing for Python keywo
  - tokenize — Tokenizer for Python so
  - tabnanny — Detection of ambiguous
  - pyclbr — Python module browser su
  - py_compile — Compile Python sour
  - compileall — Byte-compile Python
  - dis — Disassembler for Python bytec
  - pickletools — Tools for pickle deve
- MS Windows Specific Services
  - msilib — Read and write Microsoft Installer files
  - msvcrt — Useful routines from the MS VC++ runtime
  - winreg — Windows registry access
  - winsound — Sound-playing interface for Windows
- Unix Specific Services
  - posix — The most common POSIX system calls
  - pwd — The password database
  - spwd — The shadow password database
  - grp — The group database
  - crypt — Function to check Unix passwords
  - termios — POSIX style tty control
  - tty — Terminal control functions
  - pty — Pseudo-terminal utilities
  - fcntl — The fcntl and ioctl system calls
  - pipes — Interface to shell pipelines
  - resource — Resource usage information
  - nis — Interface to Sun's NIS (Yellow Pages)
  - syslog — Unix syslog library routines

Podstawy programowania w języku Python

Struktura najprostszego skryptu

```
import sys

print("argumenty:")

for arg in sys.argv[0:]:
    print(arg)
```

import modułu systemowego

wydruk tekstu w oknie konsoli

pętla drukująca argumenty skryptu

skrypt został napisany w języku o wersji Python3

py3.10

```
E:\>
E:\> python310 e:\works\skrypt.py 123 abc "witaj swiecie"
argumenty:
e:\works\skrypt.py
123
abc
witaj swiecie

E:\>
```

Leksyka języka Python

- Komentarz liniowy rozpoczyna się od znaku `#`, a blokowy zaczyna się i kończy znakami `"""` ale jak spacja, \t, \n itp. rozdziela leksemy
- Literał braku wartości: **`None`**
- Literały logiczne typu `bool`: **`True`**, **`False`**
- Literały całkowite typu `int`, np.: `-1`, `0`, `+1`, `123`
- Literały rzeczywiste typu `float`, np.: `0.0`, `3.1`, `-0.7`, `1e-30`
- Literały zespolone typu `complex` mają część urojoną zakończoną literą `j`, np.: `1+2j`, `3.14+2.7j`
- Brak literałów znakowych typu `char`
- Literały tekstowe krótkie typu `str`, np.: `"abc"`, `'Jan Pac\n'`

Uwaga: Teksty surowe (ang. *raw string*) bez interpretacji znaków ucieczki rozpoczynają się od litery `r`, np.: `r"abc \n\t\r\b xyz"`

- Teksty długie rozpoczynają się i kończą znakami `"""` lub `'''`
- Wszystkie znaki tekstów są reprezentowane w kodzie Unicode

Nazwy elementów skryptu

- Zasady dot. nazw elementów skryptu są takie same jak dla C/C++
- Nazwy są etykietą danych, funkcji, klas, pól i metod, modułów itd.
- Pewne nazwy są słowami kluczowymi używanymi w konstrukcjach składniowych i stąd są zastrzeżone; oto lista słów dla Python3:

```
False, await, else, import, pass, None, break, except,
in, raise, True, class, finally, is, return, and,
continue, for, lambda, try, as, def, from, nonlocal,
while, assert, del, global, not, with, async, elif, if,
or, yield
```

podział słów na kategorie

- Stałe i wyrażenia (True, False, and, or, not, is, in)
- Instrukcja warunkowa (if, elif, else)
- Instrukcje iteracyjne (for, in, else, while, break, continue)
- Generatory (for, in, if, yield)
- Funkcje i metody (def, return), wyrażenia lambda (lambda)
- Klasy (class)
- Moduły (import, from)
- Wyjątki programowe (assert, raise, try, except, else, finally)

Obiekty danych

- Każda dana reprezentowana jest przez obiekt w pamięci systemu
- Obiekty danych posiadają swoją tożsamość, typ oraz wartość
- Tożsamość danej jest odpowiednikiem identyfikatora zmiennej w języku C/C++, stąd podobnie traktujemy ją jak adres w pamięci
- Tożsamość danej zwracana jest przez funkcję `id`, np. dla liczby:

```
print( id(123) )
```

- Typ danej jest zbiorem wartości, jakie obiekt danej posiada, wraz ze operacjami, jakie można wykonywać na danych
- W języku Python typ danej należy traktować jak pewną klasę, co podobne jest do języka C# i systemu typów w platformie .NET
- Typ danej zwracany jest przez funkcję `type`, np. dla liczby:

```
print( type(123) )
```

- Wartość danej jest pewnym literałem zapisanym w skrypcie lub bardziej złożonym składniowym konstruktem literalnym (np. lista literałów) lub jest wynikiem ewaluacji pewnego wyrażenia

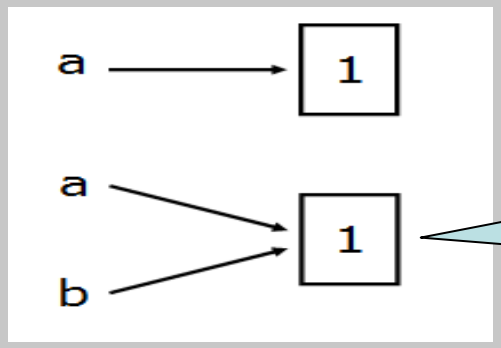
Przetwarzanie danych

- Obiekt danej, gdy pojawi się po raz pierwszy w skrypcie, jest automatycznie alokowany w pamięci (brak operatora **new**)
- Obiekt danej jest dostępny w skrypcie przez swój literał (konstrukt), wyrażenie lub pośrednio przez nazwę, a nawet wiele różnych nazw
- Obiekt danej niedostępny w skrypcie poprzez nazwę może być automatycznie usuwany z pamięci przez GC (brak operatora **delete**)
- Nazwa obiektu jest jedynie jego etykietą i w trakcie przetwarzanie skryptu jej powiązanie z obiektami może dowolnie ulegać zmianom
- Wartość pojedynczego obiektu danej jest na ogół niezmienna, co dotyczy np. liczb, tekstów, krotek, ale listy danych są już zmienne
- Mutacyjność (zmiana) danych w pamięci zależy zatem od ich typów
- Dla obiektów danych typu niemutacyjnego wirtualna maszyna Python wymusza współdzielenie unikatu z licznikiem dowiązań
- Zmiana wartości unikatu zachodzi w/g idiomu: kopiuj-przy-zapisie
- Obiekty typu mutacyjnego nie podlegają „unikatowieniu” w pamięci

Mutacyjność danych prostych

- Współdzielenie w pamięci unikatu danej typu prostego, np.:

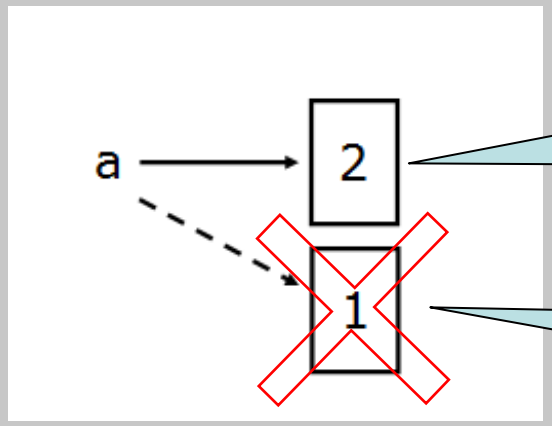
```
a = 1
b = 1
```



dwie różne nazwy dają dostęp do jednego obiektu stąd jego licznik dowiązań wynosi dwa

- Zmiana dowiązania nazwy do danych prostych, np.:

```
a = 1
a = 2
```



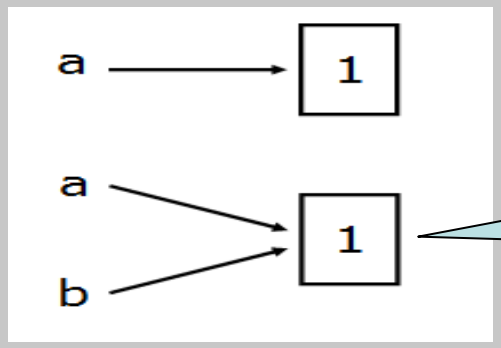
nazwa zmienia dowiązanie na nowy obiekt z licznikiem równym jeden

stary obiekt nie ma dowiązań i jego licznik to zero stąd GC go usuwa

Mutacyjność danych prostych

- Współdzielenie w pamięci danej prostej poprzez dowiązanie, np.:

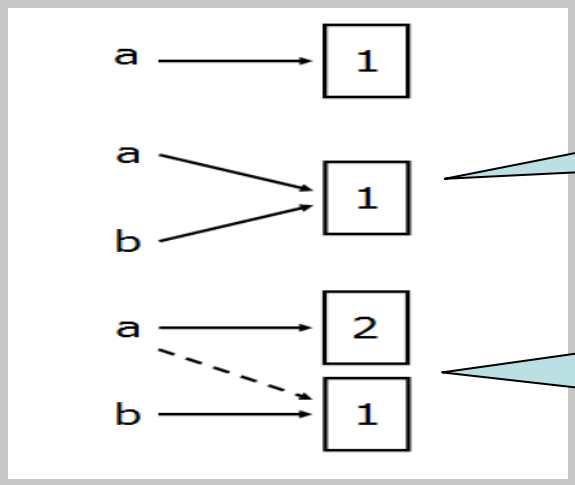
```
a = 1
b = a
```



jeden obiekt ma dwie jawnie dowiązane nazwy stąd jego licznik to dwa

- Zmiana wartości danej współdzielonej w/g kopiuj-zapisz, np.:

```
a = 1
b = a
a += 1
```



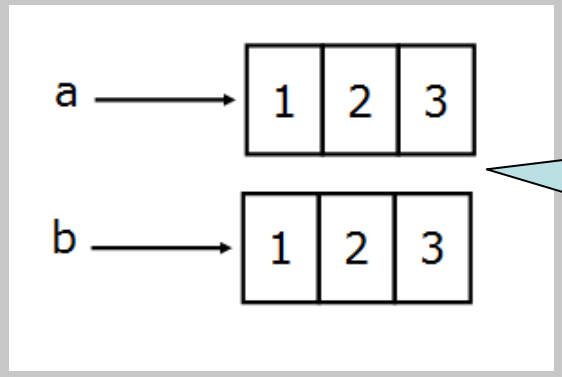
obiekt współdzielony przez dwie nazwy

nowa wartość zapisana jest w kopii obiektu, stąd współdzielenie znika

Mutacyjność danych złożonych

- Odosobnienie w pamięci duplikatów danej typu złożonego, np.:

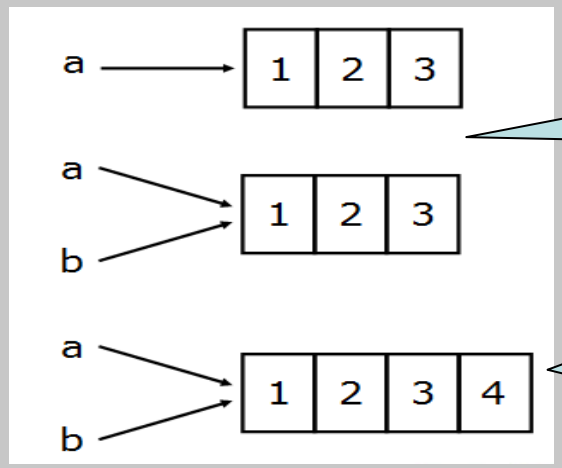
```
a = [1,2,3]
b = [1,2,3]
```



dwa identyczne obiekty o różnych nazwach z licznikami jeden

- Zmiana wartości danej złożonej ze współdzieleniem, np.:

```
a = [1,2,3]
b = a
a.append(4)
```



jeden obiekt o jednej lub wielu nazwach

modyfikacja obiektu nie skutkuje zmianą współdzielenia

Operatory i wyrażenia

- Operacje arytmetyczne o wyniku typu numerycznego:

```
**   - potęgowanie
*    - mnożenie
/    - dzielenie
//   - dzielenie z zaokrągleniem w dół
%    - reszta z dzielenia
+    - dodawanie
-    - odejmowanie
```

Przykład: Zwykłe dzielenie `3/2` daje `1.5`, zaś dzielenie `3//2` daje `1`

- Operacje relacji i logiczne o wyniku typu logicznego:

```
<, <=, >, >=    - nierówności
==, !=          - równości
is, is not      - tożsamości
in, not in      - zawierania
```

```
and - koniunkcja
or  - alternatywna
not - negacja
```

Przykład: Nierówność `a<b>c` to wyrażenie: `(a<b)` **`and`** `(b>c)`

Przykład: Tożsamość `123` **`is`** `56` to wyrażenie: `id(123)==id(56)`

Przykład: Zawieranie `2` **`in`** `[1,2,3]` to zdanie „[1,2,3] contains 2”

Operatory i wyrażenia

- Operacje bitowe o wyniku typu całkowitego:

```
~          - negacja bitowa
<< >>      - przesuniecie bitów w lewo i w prawo
&          - iloczyn bitowy AND
^          - rożnica bitowa XOR
|          - suma bitowa OR
```

Przykład: Przesunięcie bitów `111<<2` daje wartość `444`

- Pozostałe ważne operacje:
  - konkatenacja tekstów, list itd., np.: `x = "witaj"+" "+"swiecie"`
  - multiplikacja list, krotek itd., np.: `x = 2*[a,b,c]`
  - indeksowanie elementu tablicy, np.: `x = tablica[5]`
  - wycinanie zakresu tablicy, np.: `x = tablica[3:(9+1):2]`
  - wywołanie funkcji, np.: `x = funkcja(1,2,3)`
  - odwołanie do składowej klasy/obiektu, np.: `x = obiekt.atrybut`
  - wyrażenie warunkowe, np.: `x = 123 if 2>1 else "abc"`
  - wyrażenie funkcyjne, np.: `x = lambda a,b: a*b`
  - przypisanie danej do nazwy jak w C/C++ , np.: `x = (nazwa:=12)`

Priorytety operatorów

- Najwyższy priorytet mają nawiasy (_) obejmujące podwyrażenie
- Kolejno niższe priorytety mają następujące operatory:

- Indeksowanie, wycinanie (slicing), wywołanie funkcji, odwołanie do atrybutu
  `x[i]  x[i:j]  f()  x.a`
- Potęgowanie
  `**`
- Jednoargumentowe
  `+x  -x  ~x`
- Mnożenia i dodawania
  `*  /  //  %`
  `+  -`
- Przesuwania bitów
  `<< >>`
- Bitowe
  `&`
  `^`
  `|`
- Relacji
  `<  <=  >  >=  ==  !=  is  is not  in  not in`
- Logiczne
  `not`
  `and`
  `or`

operacje o tym samym priorytecie wykonywane są od lewej do prawej za wyjątkiem potęgowania i przypisania

- Najniższe priorytety mają operatory: warunkowy **if-else**, funkcyjny **lambda** oraz przypisania (**:=**)

Struktury danych

- Obok danych typu prostego można również przetwarzać obiekty o bardziej złożonej strukturze wewnętrznej w postaci kolekcji danych
- W ogólności kolekcje przyjmują postać sekwencji, mapy lub zbioru
- Sekwencje są tablicami, które mogą być indeksowane liczbą całkowitą poprzez operator `[_]`, np. listy, krotki, zakresy, teksty
- Mapy przechowują dane haszowane, do których dostęp tak jak w słowniku `dict` odbywa się poprzez indeksowanie kluczem
- Zbiory takie jak `set` przechowują jedynie unikatowe dane, które nie są uporządkowane, stąd kolekcji tej nie można indeksować
- Oprócz kolekcji wbudowanych w język można również stosować klasy kolekcji bibliotecznych, np.: `array`, `deque`, `Counter`, ...
- Kolekcje mogą być typu mutacyjnego lub nie – jak typy proste
- Kolekcje mogą być konstruktem literalnym, np. lista literałów, lub też mogą być tworzone poprzez konstrukcję obiektu z klasy kolekcji
- Liczność danych w kolekcji zwraca funkcja: `len(kolekcja)`

Struktury danych

- Lista danych jest mutacyjną sekwencją i ma wbudowany do języka typ `list` oraz konstrukt literalny, może zawierać dane różnych typów – nawet inne listy, uporządkowane w kolejności wstawiania, a indeksowanie jej elementów rozpoczyna się od zera, np.:

```
lista = [ [1, 2.0], "trzy", ['a', 'b'], True ]
print(lista); print(len(lista)); print(lista[0])
```

- Można wykonywać pewne operacje kontenerowe na listach, np.:

```
a = [1,2,3]
b = [5,6]
c = a+2*b
print(c==[1,2,3,5,6,5,6])
```

- Inne operacje są wykonywane poprzez metody klasy listy, np.:
  - append – dodaje element do listy
  - clear – usuwa wszystkie elementy
  - count – zlicza elementy o wskazanej wartości

Struktury danych

- Użycie metod klasy dla obiektu listy wymaga operatora odwołania do atrybutu obiektu, np.:

```
a = [1,2,1,3,5]
a.append(6)
print(a.count(1))
a.clear(); b = []
print(a==b)
```

- Krotki danych są listami, ale o typie niemutacyjnym `tuple`, a ich konstrukt zawiera przecinki bez nawiasów – nawiasy okrągłe stosowane są jedynie w przypadkach niejednoznaczności, np.:

```
krotka = '1', 2.0, 3, "cztery"
print(krotka); print(len(krotka))
print(krotka.index("cztery"))
print( (), 1, (2,3), 4 )
```

Struktury danych

- Słowniki danych o typie mutacyjnym `dict` są listami unikatowych par o postaci `klucz: wartość`, wstawianych w nieokreślonym porządku, a ich konstrukt objęty jest nawiasami klamrowymi, np.:

```
mapa = { 'a': 123 }
mapa['a'] = 567
mapa['b'] = 234
print(mapa); print(len(mapa))
x = {}; print(x.get('a'))
```

- Zbiory danych o typie mutacyjnym `set` są nieuporządkowane i zawierają unikaty wyłącznie o typie niemutacyjnym (np. liczby, teksty), a ich konstrukt objęty jest nawiasami klamrowymi, np.:

```
zbior = { 12, 1.3, 'a', 'b' }
zbior.add("piec")
print(zbior); print(len(zbior))
```

Struktury danych

- Udostępnienie i przetwarzanie elementów zbioru odbywa się jedynie poprzez następujące operacje mnogościowe i metody klasy – brak jest operacji indeksowania:

– suma: |
– przecięcie (część wspólna): &
– różnica: –
– dopełnienie (różnica symetryczna): ^

– add – dodaje wskazany element
– remove – usuwa wskazany element
– discard – usuwa wskazany element
– clear – usuwa wszystkie elementy

- Zawieranie w zbiorze zapisujemy jako operację z operatorem **`in`**
- Zbiór pusty zapisujemy jako konstrukcję obiektu klasy: `set()`
- Przykłady przetwarzania danych w zbiorach:

```
A = { 1, 4 }; B = { 4, 7 };
A | B     # { 1, 4, 7 }
A - B     # { 1 }
A ^ B     # { 1, 7 }
print( 123 in A )
```

```
A & B     # { 4 }
B - A     # { 7 }
```

Struktury danych

- Zakresy danych o typie niemutacyjnym `range` są zwykłymi listami liczb całkowitych wyliczanymi (generowanymi) od pewnego startu do stopu-krok co pewien krok, np.:

```
zakres = range(3, 10, 2)
print(zakres); print(len(zakres))
print(zakres[0]); print(zakres[-1])
```

- Dane listy również można proceduralnie generować w pewnym zakresie stosując specjalne wyrażenie kompletności, np.:

```
zakres = range(10)
lista = [ 2*a for a in zakres if (a%2==0) ]
print(lista); print(len(lista))
print(lista[0]); print(lista[-1])
```

- Podobnie dla zbioru ujęcie wyrażenia kompletności przez nawiasy klamrowe powoduje generowanie jego elementów

Instrukcje proste

- Wiersz skryptu z kompletną instrukcją nie ma znaku `;` (terminatora)
- Zapisu instrukcji w kilku liniach tekstu ma znak `\` jako separator
- Zapis kilku instrukcji w jednej linii wymaga znaku `;` jako separatora
- Najprostsza instrukcja to instrukcja przejścia **`pass`** powodująca natychmiastowe przejście interpretatora do następnego wiersza
- Instrukcja wyrażeniowa to pojedyncze wyrażenie, które podlega ewaluacji, a otrzymana wartość jest ignorowana w trybie wsadowym lub drukowana w trybie konwersacyjnym tuż za linią interpretacji
- Instrukcja przypisania dowiązuje nazwę do danej będącej wynikiem ewaluacji wyrażenia po prawej stronie znaku =, przy czym obok zwykłego przypisania są również rozszerzone, stąd w instrukcjach tych występują separatory:

```
=
**= *= /= //= %= += -=
```

Przykład: Przypisanie `a=b=10` to dwie instrukcje: `a=10;b=10`

Przykład: Przypisanie `a += 2` to instrukcja; `a = a+2`

Instrukcje proste

- W instrukcji przypisania po lewej stronie separatora ze znakiem = zamiast nazwy może być wyrażenie, którego wynikiem powinien być element w obiekcie typu mutacyjnego, wtedy jedynie wartość tego obiektu ulegnie zmianie, a jego licznik dowiązanych nazw pozostanie bez zmiany, np.:

```
x = [1,2,3]
print(x)
x[1] = 6*7+8
print(x)
```

```
x, y, z = [1,2,3]
print(x)
print(y)
print(z)
```

uwaga: przypisanie obiektu do ciągu (krotki) nazw to osobne dowiązania

- W pozostałych przypadkach wszystkie instrukcje przypisania oraz operacja przypisania := zwiększają licznik dowiązanych nazw
- Licznik ten można też jawnie zmniejszyć używając prostą instrukcję odwiązania `del x`, co może powodować usunięcie obiektu danej z pamięci przez GC, gdy jego licznik dowiązanych nazw osiągnie zero (pewne podobieństwo do operatora **delete** ze języka C++ ?)

Instrukcje strukturalne

- Brak instrukcji grupującej blok kodu (w C {} , w Pascal **begin-end**), a zamiast tego stosujemy wcięcia, tj. spacje (np. 4) lub tabulację
- Głębokość wcięcia daje stopień zagnieżdżenia bloków, np.:

```
def fibo(n):
    a, b = 1, 0
    for i in range(1, n):
        a, b = a + b, a
    return a
```

- Instrukcje strukturalne z blokiem kodu mogą być zapisane w kilku liniach tekstu bez jawnego separatora \ podobnie jak komentarze blokowe i teksty długie oraz krotki, listy, zbiory itp.
- W odróżnieniu od konstrukcji składniowych z wyraźnym znakiem początku i końca w postaci terminatorów jak """, (), [], {} itp. wieloliniowy zapis instrukcji strukturalnej z wciętym blokiem kodu wymaga w trybie powłoki zakończenia nową linią pustą

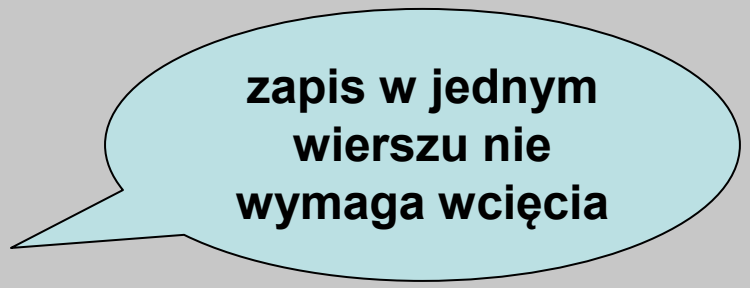
Instrukcje strukturalne

- Instrukcja warunkowa pozwalająca podejmować decyzję w skrypcie na podstawie warunków logicznych posiada ogólną składnię:

```
if <warunek1>:
    <instrukcja1>
elif <warunek2>:
    <instrukcja2>
else:
    <instrukcja3>
```

- Przykład wieloliniowego zapisu instrukcji warunkowej z wcięciami:

```
if x>0:
    print("plus")
elif x<0:
    print("minus")
else: print("zero")
```



Instrukcje strukturalne

- Instrukcja iteracyjna pozwalająca powtarzać kod w skrypcie zgodnie z logicznym warunkiem kontynuacji posiada ogólną składnię:

```
while <warunek>:
    <instrukcja>
```

- W kodzie powtarzanym przez pętlę może być prosta instrukcja **`break`** lub **`continue`** o działaniu podobnie jak w języku C
- Przykład skryptu z instrukcją iteracyjną:

```
i = 9
while i>0:
    x = 10**i
    print(i,") x =",x)
    i = i-1
print("i =",i)
```

instrukcja przed pętlą

blok kodu powtarzany przez pętlę while

instrukcja za pętlą

Instrukcje strukturalne

- Instrukcja zakresowa pozwalająca wykonać kod dla zakresu danych z kolekcji lub generowanych posiada następującą ogólną składnię:

```
for <zmienna> in <kolekcja lub generator>:
    <instrukcja>
```

- Przykłady skryptu z instrukcją zakresową:

```
for n in [1,2,3]: print(n)
for x in (123,"abc"): print(x)
for i in range(10,100,5):
    x = i**2
    print(i,") x =",x)
print("i =",i)
```

pętle po całej kolekcji danych – liście, krotce

dane generowane ze zakresu od 10 do 95 co 5

zmienna pętli jest dostępna za pętlą

Instrukcje strukturalne

- Obok predefiniowanych generatorów takich jak funkcja `range` dla instrukcji zakresowej można również stosować nowe jako wynik specjalnego wyrażenia generacji o ogólnej składni:

```
( <wyrażenie> for <zmienna> in <kolekcja> if <warunek> )
```

- Przykład skryptu z własnym generatorem:

```
gen = ( 2*a for a in [1,2,3,4,5] if a%2!=0 )
for i in gen:
    x = i**2
    print(i,") x =",x)
print(gen)
```

pętla zakresowa po generowanych danych

generator jako wyrażenie nie może być drukowany zamiast jest drukowany opis pamięci wewnętrznego obiektu

Obsługa wyjątków

- Prostą instrukcją do kontroli błędów w skrypcie są asercje pomijane, gdy interpreter jest w trybie optymalizacji (opcja –O), np.:

```
assert b!=0, "dzielenie przez zero"
x = a/b
```

- Asercja powoduje wysłanie wyjątku typu `AssertionError`, o ile predefiniowana stała `__debug__` ma wartość **`True`**
- Strukturalna instrukcja nadzorowania sytuacji wyjątkowych i obsługi wyjątków wysyłanych w skrypcie posiada ogólną składnię:

```
try:
     <instrukcja>
except <wyjatek> as <nazwa>:
     <instrukcja>
else:
     <instrukcja>
finally:
     <instrukcja>
```

- Wyjątek jest tu typem predefiniowanym lub też może być nazwą klasy dziedziczącej po wyjątku bazowym o typie `Exception`

Obsługa wyjątków

- Prosta instrukcja wysyłania wyjątku posiada ogólną składnię:

```
raise <wyjatek> from <wyjatek>
```

- Wyjątek jest tu nazwą typu lub konstrukcją z ciągiem argumentów, co powoduje utworzenie niejawnego obiektu klasy wyjątku
- Podanie klauzuli **from** umożliwia określenie przyczyny wyjątku będącej również wyjątkiem, ale poprzednio obsługiwanym
- Przykład skryptu z wtórnym wysyłaniem wyjątku:

```
try:
    print(1/0)
except Exception as przyczyna:
    raise RuntimeError("Blad") from przyczyna
```

niejawne wysłanie wyjątku przez środowisko

- Wysłany wyjątek nie obsłużony w skrypcie zostanie wyświetlony przez interpreter w konsoli/powłoce wraz ze jego przyczyną

Obsługa wyjątków

- Przykład skryptu z wysyłaniem i obsługą wyjątku:

```
try:
    x = int(input("x = "))
    if x == 0: raise ValueError
except ValueError:
    print("Bladna wartosc")
else:
    y = 1/x
    print("1/x =",y)
finally:
    print("-koniec-")
```

Obsługa wyjątków

- Przykład skryptu z asercją i obsługą wyjątku:

```
a = 12
try:
    dana = input("podaj liczbe: ")
    b = int(dana)
    assert b!=0, "dzielenie przez zero"
    x = a/b
    print("wynik dzielenia:",x)
except ValueError:
    print("Bledny typ danej")
except AssertionError as blad:
    print("Blad:",blad)
```

Definicja funkcji

- Definicja funkcji w skrypcie musi być poprzedzona słowem **`def`** ze nazwę funkcji i ciągiem jej parametrów w nawiasach okrągłych
- W bloku funkcji może być instrukcja prosta **`return`** z rezultatem
- Do wywołania funkcji służy operator `(_)` z ciągiem argumentów
- W ciągu argumentów funkcji mogą być argumenty pozycyjne, nazwane, domyślne oraz zmienne – nienazwane lub nazwane
- Argumenty z wartością domyślną można pominąć w wywołaniu
- Argumenty nazwane można podawać w dowolnej kolejności
- Argumenty pozycyjne są kojarzone z parametrami wg ich kolejności
- Argumenty zmienne nienazwane stają się krotką, a nazwane - mapą
- Nazwy funkcji nie można przeciążać jak w C++ lub C#, ale funkcje mogą być wywoływane rekurencyjnie
- Funkcja jest pewnym obiektem w pamięci, a jej pierwotną nazwę można przypisywać do innej nazwy, co daje zwykłe dowiązywanie nazw do obiektu; funkcja może być wołana poprzez wtórną nazwę

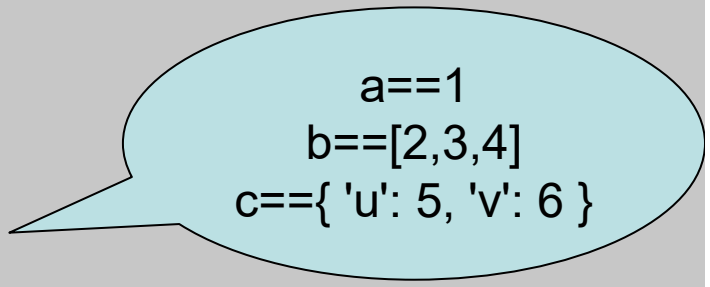
Definicja funkcji

- Przykłady definicji i wywołania funkcji ze zwykłymi argumentami:

```
def funkcja(a,b,c=33):
    print(a,b,c)
    return a+b+c
funkcja(11,22)    # c==33
funkcja(11,22,33)
y = funkcja(11,c=33,b=22)   # a==11,b==22,c==33
print(y)    # 11+22+33
```

- Przykłady definicji i wywołania funkcji ze zmiennymi argumentami:

```
def funkcja(a,*b,**c):
    print(a,b,c)
y = funkcja(1, 2,3,4, u=5, v=6)
print(y)    # None
```



Definicja funkcji

- W bloku funkcji może być prosta instrukcja **`yield`** z wartością danych kooperatywnie generowanych przez funkcję, co można wykorzystać jako złożony generator własny w instrukcji zakresowej
- Przykłady definicji i wywołania funkcji generatora danych:

```
def gen(x):
    a, b = 0, 1
    while a < x:
        yield a
        a, b = b, a+b

for i in gen(100):
    print(i)
```

zawiesza wykonywanie funkcji i dostarcza wartość wyrażenia jako kolejny rezultat funkcji, po czym wznawia wykonywanie kodu funkcji

pętla zakresowa po generowanych danych

Definicja klasy

- Definicja klasy jest poprzedzona słowem **`class`** z nazwą klasy oraz listą baz ujętych w nawiasy okrągłe; domyślną bazą jest `object`
- W bloku klasy podaje się pola statyczne z wartością początkową oraz definicje jej metod, instancyjnych lub statycznych
- Każda metoda instancyjna musi mieć jeden parametr o zwyczajowej nazwie `self`, który jest kojarzony z obiektem klasy
- Pola instancyjne wstrzykuje się dynamicznie do obiektu poprzez jego nazwę lub poprzez parametr `self` w metodzie instancyjnej
- W bloku klasy można również zdefiniować metody statyczne, co jednak wymaga użycia poprzedzającego dekoratora `@staticmethod`
- Brak jest konstruktora obiektów klasy jak w C++ lub C#, zamiast definiuje się specjalną (magiczną) metodę inicjującą o zastrzeżonej nazwie `__init__`; domyślnie metoda ta nie ma parametrów
- Inne metody magiczne służą jak operatory w C++ lub C#, np. metoda `__call__` daje obiekt z operatorem wywołania funkcji

Definicja klasy

- Obiekt jako instancja klasy tworzony jest w pamięci poprzez podanie nazwy klasy oraz ciągu argumentów w nawiasach, które kojarzone są parametrami metody inicjującej
- Statyczne składowe klasy są dostępne poprzez operator kropki z nazwą klasy, zaś składowe instancyjne wymagają nazwy obiektu
- Brak jest w klasie sekcji prywatnej i chronionej jak w C++ lub C# - wszystkie składowe klasy są publiczne, chyba że nazwę składowej poprzedza znak ___ co daje element prywatny lub znak _ co daje element chroniony, jednakże elementy te mogą być dostępne (!)
- Brak jest metod wirtualnych jak w C++ lub C#, zamiast korzysta się z dynamicznego powiązania metody wywołanej na rzecz obiektu niewidocznej klasy zgodnie z programowaniem dynamicznym
- Brak jest klas abstrakcyjnych jak w C++ oraz abstrakcyjnych interfejsów jak w C#, ale możliwe jest dziedziczenie wielobazowe podobnie jak w C++

Definicja klasy

- Przykłady definicji klasy i tworzenia jej instancji:

```
class Osoba:
    miasto = "Katowice"
    def __init__(self,im,naz):
        self.imie = im
        self.nazwisko = naz
    def wyswietl(self):
        print(Osoba.miasto,": ",\
        self.imie," ",self.nazwisko)

obiekt = Osoba("Jan", "Kowalski")
obiekt.wyswietl()
```

pole statyczne

konstruktor

pola instancyjne

metoda instancyjna

utworzenie obiektu klasy

odwołanie do składowej obiektu

Definicja klasy

- Przykłady definicji klasy z metodami magicznymi:

```
class Funktor:
    def __init__(self,a,b):
        self.a = a
        self.b = b
    def __call__(self,x):
        return self.a*x+self.b

f = Funktor(1.1,2.2)
y = f(3.3)
print(y)
```

instancyjna metoda dla operatora wołania

utworzony obiekt jest wywoływalny i zachowuje się jak funkcja

Definicja klasy

- Przykłady dziedziczenia klas i polimorfizmu dynamicznego:

```
class Figura:
    def __init__(self,r):
        self.obwod = r
    def powieksz(self,dr):
        self.obwod += dr

def Obrazek(fig):
    fig.rysuj()
    fig.powieksz(0.1)
    fig.rysuj()

class Kwadrat(Figura):
    def __init__(self,a):
        Figura.__init__(self,4*a)
        self.bok = a
    def powieksz(self,da):
        Figura.powieksz(self,4*da)
        self.bok += da
    def rysuj(self):
        print("< kwadrat o boku",self.bok,"i obwodzie",self.obwod,">")

f = Kwadrat(1.2)
Obrazek(f)
```

bazowa definicja dynamicznych metod do pokrycia

dynamiczne wywołanie metod na rzecz nieznanego obiektu

klasa pochodna

pokrycie lub nowa definicja dla metod dynamicznych

utworzenie obiektu kompletnego

Skrypty, moduły i pakiety

- Skrypt języka Python jest jego programem źródłowym
- Przyjmuje postać ciągu wierszy, czyli linii logicznych, na które składają się kolejne linie tekstowe, tj. linie fizyczne, zakończone znakiem końca linii `\n`
- W wierszach skryptu języka poza komentarzami i spacjami są zapisane główne instrukcje wykonujące algorytmy, dyrektywy dla interpretera, pomocnicze deklaracje lub definicje nowych elementów
- Definicje to złożone, blokowe instrukcje rozpoczynające się od **`def`**, których przeznaczeniem jest dodanie nowego elementu do skryptu, tj. funkcji, klasy – zmienne jako nazwane dane są wprost używane
- Dyrektywy i deklaracje przyjmują postać prostych instrukcji języka o specjalnym niealgorytmicznym przeznaczeniu, np. importowanie modułów do skryptu, modyfikowanie zasięgu nazw w zakresach

Uwaga: W skrypcie obok zdań języka mogą też być polecenia powłoki **python**, np.: help(), quit() albo powłoki **ipython**, np.: %run, %save, %colors służące do prostych operacji, a także %%timeit <wyrazenie> do pomiaru czasu wykonania wyrażenia

Skrypty, moduły i pakiety

- Moduł języka Python to skrypt w pliku tekstowym *.py|pyw lub też jest to plik binarny z kodem beta skryptu źródłowego (*.pyc|pyo)
- Modułem może też być plik z kodem maszynowym (*.pyd ~ .dll|so) pochodzącym od innego języka programowania, np. C/C++
- W trakcie interpretacji moduł reprezentowany jest przez nazwę główną jego pliku, tj. bez końcówek .* - kropka ma inne znaczenie
- Pliki modułów w środowisku Python gromadzone są w katalogu **<pyroot>\Lib** lub w jego podkatalogach zwanych pakietami

Uwaga: Każdy taki pakietowy podkatalog musi zawierać (może być pusty) skrypt o nazwie: '__init__.py'

- Biblioteka standardowa jest kolekcją pakietów i modułów, stąd stanowi komponent środowiska - gramatyka języka zawiera jedynie kilka wbudowanych stałych, typów, funkcji, wyjątków oraz atrybuty
- Podstawowa instalacja środowiska zawiera obok standardowego interpretera i innych narzędzi także pakiety biblioteki standardowej
- Pozostałe pakiety instalowane są dodatkowo po pierwotnej instalacji i stąd stanowią dowolne rozszerzenie środowiska

Importowanie modułów i pakietów

- W trakcie interpretacji bieżącego skryptu można zaimportować osobne moduły lub moduły z pakietów środowiska, aby uzyskać dostęp do elementów modułu, tj. danych, funkcji, klas i ich instancji
- Interpreter wykonuje to wskutek dyrektywy importowania, która jest jedynym środkiem języka Python umożliwiającym dynamiczne załadowanie modułu i powiązanie go z bieżącym skryptem, np.:

```
import modul
 # importowanie nieskatalogowanego pliku `modul.py|pyc|pyo|pyd` jako alias `modul`
import pakiet.modul1 as pm1
 # importowanie pliku `modul1.*` z katalogu `pakiet` jako alias `pm1`
from pakiet import modul2 # zamiast pakietu mogą być znaki `.` dla kat. bieżącego, nadkatalogu...
 # importowanie pliku `modul2.*` z katalogu `pakiet` jako alias `modul2`
from pakiet.modul3 import fun # zamiast elementu może być znak `*` dla publicznych elementów
from pakiet.modul3 import fun as pm3fun
 # importowanie funkcji `fun` z pliku `pakiet/modul3.*` jako alias `fun` lub `pm3fun`
```

- Dostęp do elementów modułu odbywa się poprzez aliasową nazwę lub ścieżkę katalogową do pakietu wraz ze operatorem kropki

Uwaga: W języku Python brak jest osobnych przestrzeni nazw jak w C++ lub C#, a zamiast stosuje się nazwy pakietów i modułów jako przestrzenie, czego przykładem jest moduł `builtins` będący w istocie przestrzenią nazw elementów wbudowanych języka

Elementy języka

- Gramatyka języka udostępnia obok leksemów i konstrukcji również elementy wbudowane, których nazw nie musimy jawnie definiować – są to zatem nazwy predefiniowane
- Przykłady zmiennych wbudowanych: __debug__, NotImplemented, Ellipsis (też … jako obiekt literalny pominięcia), copyright, license
- Przykłady typów wbudowanych: bool, int, float, complex, list, tuple
- Przykłady funkcji wbudowanych: id, type, print, len, dir, abs, max
- Przykłady wyjątków wbudowanych: BaseException, AssertionError, SystemExit, Exception, OverflowError, TypeError, ValueError
- Wszystkie te elementy należą do przestrzeni modułu `builtins`, co można jawnie importować, gdy wystąpi przysłonięcie własną nazwą w skrypcie jakieś predefiniowanej nazwy - jest to w pełni dozwolone
- Przykładem innych elementów języka o predefiniowanych nazwach są atrybuty: __doc__, __name__, __self__, __module__, __func__... które są emitowane przez interpreter jak makro-symbole we C/C++

Zakresy skryptu i zasięg nazw

- Nazwy elementów skryptu powiązane są z pewnym zakresem jego kodu, przy czym zasięg ich widoczności i dostępności rozciąga się od punktu ich pierwszego wystąpienia w kodzie do końca zakresu
- W języka Python wyróżniamy zakres globalny, lokalny oraz klasy
- Zakres globalny występuje poza definicjami funkcji i klas i dotyczy całego skryptu/modułu, natomiast zakres lokalny występuje tylko w definicji funkcji - blok w instrukcji strukturalnej nie jest zakresem
- Każdy zakres kodu w skrypcie może zawierać dowolne instrukcje języka, a nawet definicje, co daje zagnieżdżanie zakresów
- Definicję klasy jako zakres należy odróżnić od zakresów globalnego i lokalnego, gdyż definicja ta obejmuje jedynie pola i metody oraz definicje klas zagnieżdżanych, do których dostęp jest przez kropkę
- Instrukcje z zakresu globalnego modułu są wykonywane także po jego zaimportowaniu, a zdefiniowane lub używane w tym zakresie nazwy elementów przenoszą swój zasięg do skryptu importujący

Zakresy skryptu i zasięg nazw

- Przykład globalnego zakresu modułu:

```
zmienna = 123 # globalne nazwy publiczne
def funkcja(x): return 2*x+1
_prywatny = -1 # globalna nazwa prywatna
print("import modulu",__name__) # kod globalny
```

- …importowanego przez skrypt:

```
import modul1 # import modułu i wykonanie globalnego kodu
from modul1 import * # lub jedynie import nazw publicznych
a = zmienna # bezpośredni dostęp do nazw publicznych
b = funkcja(5)
# c = _prywatny # w przypadku nazwy prywatnej możliwy...
c = modul1._prywatny # ...jest jedynie dostęp pośredni
print("w skrypcie",__name__)
print(a,b,c)
```

Zakresy skryptu i zasięg nazw

- Atrybut `__name__` to nazwa modułu podlegającego importowi, ale dla bieżącego skryptu atrybut ten zawiera tekst "__main__"
- Atrybut ten pozwala dla modułu zaniechać wykonywania kodu głównego w jego zakresie globalnym, a wymusić to jedynie dla bieżącego skryptu, np.:

```
zmienna = 123
def funkcja(x): return 2*x+1
_prywatny = -1
def main():
    print("kod glowny skryptu")
if __name__ == "__main__": # kod globalny
    main()
```

- Plik ten może pełnić jednocześnie rolę importowalnego modułu lub wykonywalnego skryptu

Zakresy skryptu i zasięg nazw

- Nazwy z zakresu globalnego nie można użyć w zakresie lokalnym, ale nazwa lokalna może przysłonić nazwę globalną, bo ma zasięg jedynie lokalny, chyba że została użyta deklaracja **`global`**, np.:

```
a = 10
def funkcja():
    global b
    b = 20
    print(b)
    global a
    print(a)
    a = 21
    print(a)
funkcja()
print(a)
print(b)
```

deklaruje nazwę w zakresie globalnym, gdzie będzie także dostępna

bez tej deklaracji nazwa globalna nie jest dostępna i dalej będzie przysłonięta przez nazwę lokalną

dostęp do nazw używanych w zakresie globalnym lub zadeklarowanych lokalnie jako nazwy globalne

Zakresy skryptu i zasięg nazw

- Podobnie zachodzi dla nazw z lokalnego zakresu nadrzędnego oraz podrzędnego, chyba że została użyta deklaracja **`nonlocal`**, np.:

```
def funkcja1():
    x = 10
    print(x)
    def funkcja2():
        global y
        y = 21
        nonlocal x
        print(x)
        x = 20
        print(x)
    funkcja2()
    print(x)
funkcja1()
print(y)
```

nazwa lokalna z zakresu nadrzędnego

nazwa dostępna również w zakresie globalnym

nazwa lokalna dostępna również w zakresie podrzędnym bez przysłaniania nazwy nadrzędnej

Dokumentacja języka

The Python Language Reference

This reference manual describes the syntax and “core semantics” of the language. It is terse, but attempts to be exact and complete. The semantics of non-essential built-in object types and of the built-in functions and modules are described in *The Python Standard Library*. For an informal introduction to the language, see *The Python Tutorial*. For C or C++ programmers, two additional manuals exist: *Extending and Embedding the Python Interpreter* describes the high-level picture of how to write a Python extension module, and the *Python/C API Reference Manual* describes the interfaces available to C/C++ programmers in detail.

- 1. Introduction
  - 1.1. Alternate Implementations
  - 1.2. Notation
- 2. Lexical analysis
  - 2.1. Line structure
  - 2.2. Other tokens
  - 2.3. Identifiers and keywords
  - 2.4. Literals
  - 2.5. Operators
  - 2.6. Delimiters
- 3. Data model
  - 3.1. Objects, values and types
  - 3.2. The standard type hierarchy
  - 3.3. Special method names

